

DLDCALab 3: Performance

Week 3

August 26, 2025

1 Introduction

In this week's lab, you shall be learning how to optimize code by exploiting the power of SIMD instructions. In a nutshell, SIMD instructions allow you to *vectorize* your code. They exploit parallelism inherent in certain workloads to boost performance.

2 Necessary tools

We shall be using gcc and g++ in order to compile C and C++ code.

Before starting the lab, ensure that the tool is installed in your system. This can be verified by running the following:

```
$ gcc
gcc: fatal error: no input files
compilation terminated.
$ g++
g++: fatal error: no input files
compilation terminated.
```

You will also need to ensure that your system has access to SIMD 256-bit registers. For this, run the following:

```
$ lscpu | grep -i avx2
... avx2 ....
```

3 Structure of submission/templates

```
your-roll-no/
|-common
  |- matrix.hpp
|- task1/
  |- Makefile
  |- mask.cpp (TODO)
  |- vector.hpp
|- task2/
  |- Makefile
  |- main.cpp
  |- naive.cpp
```

```
| - optim.cpp (TODO)
| - image-convolve.cpp
| - image-process.py
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of `your-roll-no.tar.gz`. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

4 Tasks

This assignment consists of two subtasks. These tasks are expected to be submitted out-lab. The first task is intended to parallelize simpler algorithms and the second task is for slightly more complicated algorithms.

Common

We have given you an implementation of a matrix class, along similar lines as the *vector* class that was given in the inlab. The matrix class has the following properties:

1. Size: It returns a pair of two 64-bit unsigned integers, representing the number of rows and the number of columns in the matrix respectively
2. Element: It takes in a pair of integers, say (i, j) as arguments and returns the address of the element of the matrix in the i^{th} row and j^{th} column.

When a matrix is initialized, the dimensions have to be specified, and it is initialized with all its elements being 0.

Task 1: Parallelize this!

The goal of this task is to show that SIMD instructions can even be used for parallelizing seemingly convoluted or difficult-to-parallelize codes. You are given code in `mask.cpp` which performs action *somewhat* similar to the forward pass of a **tiny** neural network, one with just 2 layers, the first having n (in the code, $n = \text{NUM_CELL}$) *cells* and the other having 1 cell.

Here is some background on the structure of the neural network:

1. Input: The input is assumed to be a k dimensional vector, in the code, the input is called `input` and k is called `INPUT_DIM`
2. Layer 1: Each cell has a k dimensional vector, let us call this vector as `layer1CellWeights`. The vector of the i^{th} cell can be accessed by `layer1CellWeights[i]`. Each cell takes in the input and outputs a singular number. Therefore, collectively from layer 1, the output is n numbers, which are represented by an n dimensional vector.
3. Layer 2: The cell does not any weights, it simply performs an arithmetic operation on the n dimensional vector output by cells of layer 1.

Given an input vector of dimension k , the neural network performs the following actions:

1. Layer 1: The output $y[i]$ of a cell i is given by the following equation

$$y[i] = \sum_{j=0}^{j < k} \max(0, w[i, j] * input[j])$$

where $w[i, j]$ is the j^{th} double stored in the i^{th} cell's weight vector, it can be accessed using `*(layer1CellWeights[i]->element(j))`

2. Layer 2: The output out of the cell is given by the following equation

$$out = \sum_{j=1}^{j < n} \max(y[j], (y[j - 1] + 1) * y[j])$$

We have already given you the naive implementation of this code. Your job is to use SIMD instructions (from Section 5) to get the code to run faster.

Task 2: One Step Ahead

In this task we will be parallelizing some slightly more complicated matrix algorithms. We have given you the naive implementation of all of these algorithms. Your job is to implement parallelized implementations of these algorithms using the cheatsheet given in section 5.

1. `outer(matrix *a, matrix *b)`: Known as the *outer product* or the *tensor product*, given a matrix a of dimensions (r_1, c_1) and a matrix b of dimensions (r_2, c_2) , the function returns a pointer to a matrix c of dimensions $(r_1 * r_2, c_1 * c_2)$ where

$$\forall i \forall j \forall p \forall q, c[i * r_2 + p, j * c_2 + q] = a[i, j] * b[p, q]$$

An application of this function is in quantum computing, where it is used to represent some computation occurring on multiple qubits

2. `convolve(matrix* a, matrix* b)`: This function implements the *convolution* operation on matrices, an overview of the operation can be seen in figure 1. In a nutshell, given matrices a of dimension (r_1, c_1) and b of dimension (r_2, c_2) such that both r_2 and c_2 are odd (this is a restriction in our algorithm, not necessarily for all convolution operations), we return a pointer to a matrix c such that

$$c[i, j] = \sum_{k=-\frac{r_2-1}{2}}^{\frac{r_2+1}{2}} \sum_{l=-\frac{c_2-1}{2}}^{\frac{c_2+1}{2}} a[i+k, j+l] * b[k + \frac{r_2+1}{2}, l + \frac{c_2+1}{2}]$$

$$\forall i \forall j, \frac{r_2-1}{2} \leq i < r_1 - \frac{r_2-1}{2}, \frac{c_2-1}{2} \leq j < c_1 - \frac{c_2-1}{2}$$

and $c[i, j] = 0$ otherwise. An application of this is in image processing, where convolution is used to blur or de-blur images. For instance, figure 2 depicts what happens when an image is taken as a matrix and convoluted with a specific matrix chosen to sharpen the image. The code used to sharpen the images is given to you and can be used using the command `python3 image-process.py <your image.png> <output image.png> image-convolve`

3. `weirdMul(matrix* a, matrix* b)`: This function is a variant of the `matmul` function with a twist,

$$\forall i \forall j, c[i, j] = \sum_k \{(a[i, k] > 0.5) ? (a[i, k]) : (a[i, k] - 1)) * b[k, j]\}$$

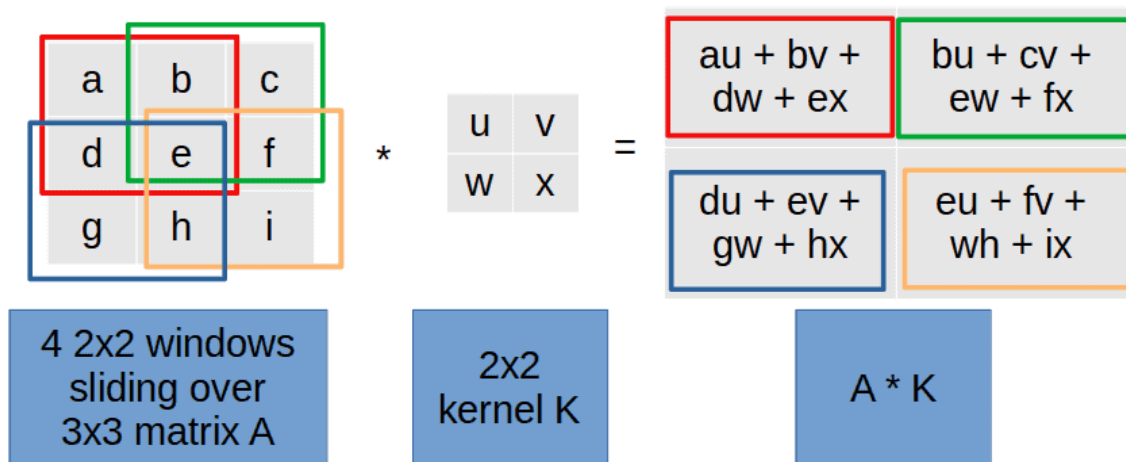


Figure 1: A figure depicting the convolution operation on matrices



(a) Some gentlemen in traditional clothes. I wonder who these folks are?

(b) Same gentlemen after convolution with a specific matrix. Do they look sharper?

Figure 2: Comparison of the original and sharpened image.

5 SIMD Instructions Cheatsheet

Instruction	Arguments	Return Value	Effect
<code>_mm256_setzero_pd</code>	None	<code>__m256d</code>	Returns 256 bits which are all zero
<code>_mm256_loadu_pd</code>	mem	<code>__m256d</code>	Loads 256 bits from mem, interprets them as doubles, and returns them
<code>_mm256_storeu_pd</code>	mem, <code>__m256d</code>	None	Stores 256 bits from the register into mem as doubles
<code>_mm256_add_pd</code>	<code>__m256d</code> , <code>__m256d</code>	<code>__m256d</code>	Returns 4 doubles packed into 256 bits, each is the sum of the corresponding input doubles
<code>_mm256_sub_pd</code>	<code>__m256d</code> , <code>__m256d</code>	<code>__m256d</code>	Returns 4 doubles packed into 256 bits, each is first double minus second double
<code>_mm256_mul_pd</code>	<code>__m256d</code> , <code>__m256d</code>	<code>__m256d</code>	Returns 4 doubles packed into 256 bits, each is the product of the input doubles
<code>_mm256_div_pd</code>	<code>__m256d</code> , <code>__m256d</code>	<code>__m256d</code>	Returns 4 doubles packed into 256 bits, each is first double divided by second double
<code>_mm256_fmadd_pd</code>	<code>__m256d</code> , <code>__m256d</code> , <code>__m256d</code>	<code>__m256d</code>	Returns 4 doubles packed into 256 bits, each is (first double * second double) + third double
<code>_mm256_max_pd</code>	<code>__m256d</code> , <code>__m256d</code>	<code>__m256d</code>	Returns 4 doubles packed into 256 bits, each is maximum between first double and second double
<code>_mm256_min_pd</code>	<code>__m256d</code> , <code>__m256d</code>	<code>__m256d</code>	Returns 4 doubles packed into 256 bits, each is minimum between first double and second double
<code>_mm256_set1_pd</code>	double	<code>__m256d</code>	Returns 4 doubles packed into 256 bits, each is having the value of the input double

Table 1: Common SIMD instructions